KuP Skill-Manager Handbook

User Manual + skillctl Reference

m3c · Scalytics — KuP Berlin training

2026-05-07

Contents

Skill-Manager – User Manual

A trustworthy supply chain for living skills, with field-feedback mechanisms. This manual tells you, **by role**, what to do – what to type, what to expect, what to avoid. The companion SKILLCTL-MANUAL.md documents every `skillctl` command.

---

0. Read this first

A **skill** is a directory of instructions, scripts, prompt templates, and references that an agent (Claude Code) loads and executes. Skills live at:

Tier	Where	Who installs
project	`<repo>/.claude/skills/<name>/`	repo owner
user	`~/.claude/skills/<name>/`	end-user
plugin	`~/.claude/plugins/cache/**/skills/<name>/`	plugin marketplace

Project beats user beats plugin (Claude Code's own resolution rule). Without governance, every skill is **arbitrary code at the LLM's privilege level**. The Skill Manager is the m3c capability that turns those directories into a signed, attested, audited supply chain.

Six things you should know before you touch the system:

1. The unit of trust is a **skill bundle** (`<name>-<version>.skb`) – a deterministic gzipped tar whose SHA-256 is the canonical `bundle_digest`.
2. The chain layers **three signatures**: author, registry, governance – each detached, named after the digest, named `<digest>.author.sig` / `.registry.sig` / `.governance.sig`.
3. The **registry** is API-only (`aims-core/flask/modules/skill_registry/`). The **CLI** is `skillctl`. The **UIs** are `skillprofile` (your personal skills) and the **CISO Console** (`/v2/ciso?tenant=<id>`).
4. Every state-changing operation has a **numbered exit code** (SPEC-0188 §11 + §17-§39 across SPEC-0201/-0202). Scripts and pre-commit hooks should branch on exit codes, not parsed log lines.
5. **Trust attaches at our boundary, never upstream.** Public hubs are discovery surfaces; bytes only enter the registry through `skillctl import-public` -> `propose` -> reviewer attestation (SPEC-0201).
6. **Every invocation is gated.** Even after install, a skill runs under a short-lived **capability token** (SPEC-0202) that bounds egress, subprocess, file I/O, and runtime. Refusals are audited.

If you only read one thing else: read SECTION 4 – The Five Lifecycle Flows. Everything else is reference.

---

1. Quick start by role

Find your role. Run the three commands. Read the linked playbook.

1.1 Author / Skill-Steward

You write skills. You ship them.

```
# 1) Verify your environment
skillctl --help                                    # CLI present
ls ~/.claude/skillctl-keys/author.key              # keypair present (else: `skillctl keygen

# 2) Iterate on a skill, then propose
skillctl propose <skill-name> --intent yellow --rationale "<what changed>"

# 3) Watch the queue
open https://aims.example.com/v2/skills/admin      # your bundle awaits a reviewer
```

Detailed playbook: §5.1 Author.

1.2 Reviewer

You sign the binding governance verdict.

```
# 1) See what's awaiting review
open https://aims.example.com/v2/skills/admin       # tab: Bundles -> filter "no_governance"

# 2) Read the bundle, run the Activation Gate (SPEC-0130 §5.1)
skillctl install <name>@<version> --home /tmp/review-sandbox --allow-yellow
diff -ur /tmp/review-sandbox/.claude/skills/<name>/ <expected>

# 3) Sign the verdict
skillctl attest <bundle-digest> --level green --rationale "<why>" \
    --reviewer-id id:reviewer@<org> --key ~/.claude/skillctl-keys/reviewer.key
```

Detailed playbook: §5.2 Reviewer.

1.3 CISO (Tenant Security Officer)

You authorize what runs inside *your* tenant.

```
# Everything happens in the console
open https://aims.example.com/v2/ciso?tenant=<your-tenant>
# Tabs you'll use:
#   Bundles            -- the inventory
#   Attestation queue  -- Approve · Block · Escalate
#   Data sources       -- per-(skill, source) [authorize|deny|pending] chips
#   Runtime (SPEC-0202) -- live tokens, [×] revoke, [a] attenuate
#   Audit              -- quarterly export PDF/CSV
```

Detailed playbook: §5.3 CISO.

1.4 Registry Operator

You own keys, identities, per-tenant infrastructure.

```
# Provision a tenant
cd m3c-tools-maintenance/INFRA/skill-registry/
./bootstrap.sh --tenant <tenant-id>                 # GCS bucket + Firestore namespace + Secret

# Rotate the registry key (overlap-publish)
open CISO-WORK/RUNBOOK-skill-registry-key-rotation.md   # follow this verbatim

# Register an author identity (v1 manual)
curl -X POST $REG/api/skills/identities \
    -H "Authorization: Bearer $ADMIN_TOKEN" \
    -d '{"id":"id:author@org","ed25519_pubkey":"<base64>","tier":"user"}'
```

Detailed playbook: §5.4 Registry Operator.

1.5 Tenant Admin

You configure the per-tenant runtime: policy, SLAs, role bindings.

```
# Edit the tenant config
$EDITOR tenants/<tenant-id>/config.yaml             # then commit
# Bind a CISO
```

```
curl -X POST $REG/api/tenants/<tenant-id>/role-bindings \
    -d '{"role":"tenant_ciso","identity":"id:ciso@<org>"}'
```

Detailed playbook: §5.5 Tenant Admin.

1.6 End-User / Skill Consumer

You install verified skills and run them.

```
# 1) Pin trust roots (one-time)
skillctl trust add --registry https://aims.example.com/api/skills \
    --pubkey ~/.config/m3c/skill-trust-roots/<reg>.pub --id <reg-id>

# 2) Install
skillctl install didactic-session@1.0.0 --tenant <your-tenant>

# 3) Run (envelope-bounded -- SPEC-0202)
skillctl run didactic-session
```

Detailed playbook: §5.6 End-User.

1.7 Auditor

You read, you reconstruct, you flag.

```
# 1) Pull a quarterly audit export from the CISO console
open https://aims.example.com/v2/ciso?tenant=<id>&tab=audit

# 2) Reconstruct a single skill's chain
curl $REG/api/skills/by-digest/<digest>                          # bundle row
curl $REG/api/skills/<digest>/signatures                         # author + registry sigs
curl $REG/api/skills/<digest>/attestations                       # all governance attestati
curl $REG/api/skills/audit-log?bundle_digest=<digest>            # chronology

# 3) Compare to local install
skillctl audit                                                   # OK / UNVERIFIED / BROKEN
```

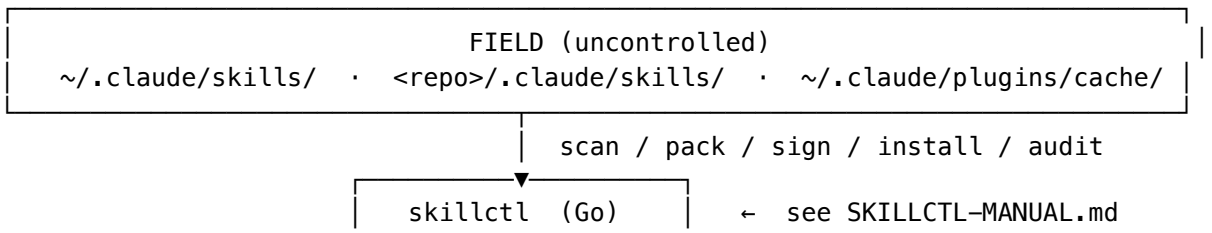
Detailed playbook: §5.7 Auditor.

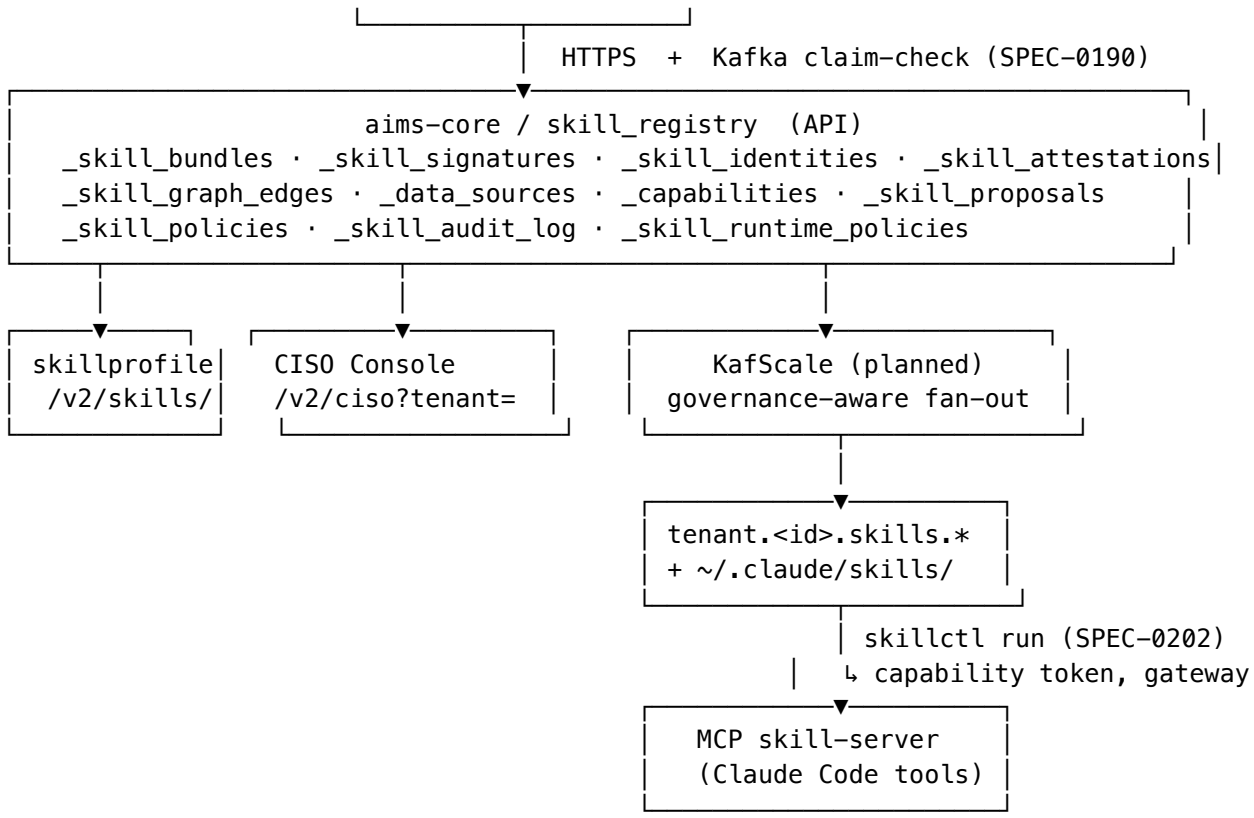
1.8 m3c Consultant (delivery role)

You take a customer Bronze->Silver->Gold (see §7 Maturity model). Your authority is operational only – you act as Author for training corpora and never as Reviewer or CISO of the customer's production skills.

---

2. The system, in one diagram





The **load-bearing files on every operator's laptop:**

Path	What	Owner
`~/.claude/skill-trust-roots.yaml`	Pinned registry public keys (which keys you accept)	End-User pins; Operator distributes
`~/.claude/skillctl-keys/author.key`	Author signing key (PEM PKCS#8 mode 0600)	Author
`~/.claude/skillctl-keys/reviewer.key`	Reviewer signing key (Q3-2026 stays registry-on-behalf-of in v1)	Reviewer
`~/.claude/skill-import-policy.yaml`	Source policy for `import-public` (SPEC-0201)	CISO/Operator
`~/.cache/m3c/imports/<sha256>/`	Stage area for upstream imports (post-airlock, pre-propose)	Author
`~/.config/m3c/skill-keys/`	Suggested keypair location from `skillctl keygen`	Anyone holding a key

If any of those files leak to git, treat as an incident. The pre-commit `gitleaks` scan from `/security-scan` catches the `.priv` extension by default.

---

3. Concepts you cannot skip

3.1 Bundle anatomy

A `.skb` is a **gzip-compressed tar with deterministic normalization** (SPEC-0188 §3.1). Two `skillctl pack` runs over the same input produce byte-identical bytes.

```
didactic-session-1.0.0.skb
├── bundle.json                          ← manifest
├── CHECKSUMS                            ← sha256 per file
├── SKILL.md                             ← the skill itself
├── helpers/
├── profiles/
└── scripts/                             ← mode 0755; everything else 0644
```

Determinism rules: lexicographic order, mtime = `1970-01-01T00:00:00Z`, tar PAX format with no extended headers, gzip header zeroed (`gzip --no-name` semantics), no `.DS_Store` / `.git` at root. `built_at` in `bundle.json` is also Unix epoch – wall-clock would break determinism; the real timestamp lives in the registry attestation.

3.2 The `bundle.json` manifest

```jsonc
{
  "schema": "m3c-skill-bundle/v1",
  "name": "didactic-session",
  "version": "1.0.0",
  "summary": "Scaffold a live training session for a specific role-track.",

  "source_repo": "kamir/.claude",
  "source_commit": "",
  "source_path": ".claude/skills/didactic-session",

  "author_governance_intent": "yellow",
  "author_governance_rationale": "Generates files; no destructive ops.",

  "depends_on": [],
  "supersedes": null,
  "derived_from": null,

  "intent": {                                          ← SPEC-0196
    "summary": "...",
    "side_effects": ["fs:write", "llm:call"],
    "destructive": false,
    "network": false,
    "subprocess": ["pandoc", "git"],
    "claims": ["skill:document-generation"]
  },
  "data_dependencies": [                               ← SPEC-0196
    {"id": "ds:filesystem/cwd", "kind": "local_fs",
     "access": "write", "scope": "<cwd>/decks/**",
     "reason": "Write scaffolded decks."}
  ],

  "runtime_envelope_required": true,                   ← SPEC-0202

  "imported_from": null,                               ← SPEC-0201

  "bundle_digest": "sha256:cd1d5c79...",
  "built_at": "1970-01-01T00:00:00Z",
  "built_by": "skillctl/1.0.0"
```

```
}
```

Two non-obvious rules:

- `author_governance_intent` is **advisory only**. Verifiers MUST NOT use it for trust decisions; the binding verdict is the most-recent signed governance attestation (SPEC-0188 §3.2 + §4.3). Author intent and signed verdict can disagree; when they do, the attestation wins.
- `intent` and `data_dependencies` are cross-validated by `skillctl pack` (SPEC-0196 §3.3). A destructive skill cannot self-claim (G); a `network=true` skill MUST declare an HTTP `data_dependency`; a write-access dependency MUST come with `intent.destructive=true`. Pack refuses with exit 18 (`intent_inconsistent`).

3.3 The trust chain

Three detached ed25519 signatures alongside the bundle:

```
didactic-session-1.0.0.skb
<digest>.author.sig          ← author signed bundle_digest
<digest>.registry.sig     ← registry signed (digest, author_sig, admission_ts, decision)
<digest>.governance.sig      ← reviewer signed (digest, level, ts, reviewer_id)
```

The verifier (`skillctl install`) checks each signature against pinned roots in `~/.claude/skill-trust-roots.yaml`. Numbered exit codes distinguish failure modes (digest mismatch / author sig invalid / registry not in roots / governance below minimum / depends_on unsatisfied / data dep denied / token expired / etc.). See §8 Exit codes.

3.4 The semantic graph

The trust graph contains **structural** edges (who signed, who admitted, who attested) AND **semantic** edges (what the skill claims to do, what data sources it touches). Stored denormalised in Firestore (`_skill_graph_edges`). Node kinds: `Skill`, `SkillVersion`, `Identity`, `Attestation`, `Capability`, `DataSource`, `Role`, `UserSkillProfile`, `Package`. Edge kinds: `signed_by`, `admitted_by`, `attested`, `supersedes`, `depends_on`, `claims_capability`, `reads_from`, `writes_to`, `mapped_to`, `used_by`, `imported_from`. Composite indexes cover all CISO-console queries in O(1) hops; the same shape ports 1:1 to KafGraph when load demands.

3.5 The runtime envelope (SPEC-0202)

Every invocation gets a short-lived signed **capability token** bound to `(bundle_digest, tenant_scope, caller_identity, envelope, expires_at=300s)`. The envelope is the **intersection** of (bundle declaration ∩ tenant policy ∩ caller request) – the issuer never grows rights. The gateway shim refuses anything outside the envelope and logs every refusal. The CISO can `[×]` revoke or `[a]` attenuate live tokens.

You don't see the envelope until you violate it, and then you see exit 30-39.

---

4. The five lifecycle flows

Flow diagrams from `ROLES.md` §2, fleshed out into the actual commands at every step.

4.1 Happy path – new skill landing in production

```
Author ─pack+sign─▶ Registry ─admit+attest─▶ Reviewer ─gov-sig─▶ CISO ─tenant-verdict─▶ End-User
```

#	Role	Command / action	Produces
1	Author	`skillctl propose <name> --intent yellow --rationale "<why>"` (runs the SPEC-0194 §6 ten-check ready-to-promote gate, then `pack` -> `sign` -> `POST /api/skills/bundles`)	author.sig + registry row
2	Registry Operator (automatic)	Server verifies the author signature against `_skill_identities`, emits the registry attestation (SPEC-0188 §4.2), writes `_skill_bundles`, `_skill_signatures`, `_skill_graph_edges`	registry.sig + admission edges
3	Reviewer	Open `/v2/skills/admin` -> Bundles -> click bundle -> read SKILL.md, scripts, intent, data_dependencies -> run Activation Gate (SPEC-0130 §5.1) -> `skillctl attest <digest> --level green --rationale "..." --reviewer-id <id> --key <path>`	governance.sig + attested edge
4	CISO	`/v2/ciso?tenant=<id>` -> Attestation queue -> Approve/Block/Escalate -> for each `data_dependency` chip click `[Authorize]` or `[Deny]`	tenant verdict + data-source authorization

#	Role	Command / action	Produces
5	End-User	`skillctl install <name>@<version>`. Verifier checks digest -> author sig -> registry sig -> governance ≥ minimum -> tenant policy -> data deps -> deps installed -> atomic move to `~/.claude/skills/`. Then `skillctl run <name>` for envelope-bounded execution	install record + Kafka invocation events

Tiebreaker: at every step the next role can refuse. Refusal exits with a numbered code; the chain halts.

4.2 Awareness path – scanned local skill becomes registry-aware

This is the SKILLOR-WORK/s1 flow, productized as SPEC-0195. Use it when you have local skills that were never packed (e.g. plugin marketplace installs, hand-rolled scripts).

`End-User —scan→ Registry —admit-from-scan→ Auditor —reads chain→ CISO —authorize data deps→ End User`

#	Role	Command / action
1	End-User	`skillctl scan --source claude --with-trust --format json > inventory.json`
2	End-User	`skillctl awareness sync --inventory inventory.json --confirm --default-attest yellow --session "skill-awareness/<host>/<date>"`
3	Registry (automatic)	One bundle per scanned skill admitted with `gcs_uri = local-aware://<digest>`. Bundles are *known* but bytes are not transferable until re-published via the Author path. Per SPEC-0196 §7 missing-intent skills get sentinel `intent.side_effects=["UNKNOWN"]` + `unverified_intent` marker edge
4	Auditor	Reads the new admissions in the audit log; flags any skill with `UNKNOWN` intent for follow-up

#	Role	Command / action
5	CISO	Sees new entries in the verdict queue with `needs declaration` badges. Until authorized, downstream consumers see `pending` chips
6	End-User	`skillctl awareness verify --session "<tag>"` reads back per-session admissions for confirmation

A full worked example (with reset-and-replay, snapshots, and verification) lives in SKILLOR-WORK/s1/USER-MANUAL.md.

4.3 Demotion path – Reviewer revokes attestation

`Reviewer –new-attest (R)→ Registry –republish current_gov→ Tenant Admin –notify→ End-User loader`

#	Role	Command / action
1	Reviewer	`skillctl attest <digest> --level red --rationale "<why>" --reviewer-id <id> --key <path>` (multiple attestations allowed; most-recent qualified one wins)
2	Registry (automatic)	Updates `_skill_attestations`; the graph's `attested` edge resolves to the new verdict on next read
3	Tenant Admin (notification)	Receives an SLA-tracked notification (SPEC-0192 §7) for any tenant bundle whose `current_governance` dropped below the tenant's `governance_minimum`
4	CISO	Either accept (block propagates) or override (re-attest (G) tenant-scoped)
5	End-User loader	Next `skillctl install` sees `governance below minimum` (exit 14) and refuses. Already-installed copies on disk still work – `skillctl audit` flags them on the next sweep; `--cleanup` removes them

4.4 Rotation path – Author rotates key

`Author –new-key→ Registry –register identity v2→ Reviewer –re-sign new bundle→ End-User upgrade`

#	Role	Command / action
1	Author	`skillctl keygen --out ~/.claude/skillctl-keys/author-v2`. Keep old key marked retired.
2	Registry Operator	`POST /api/skills/identities` with the new pubkey; identity row carries both pubkeys during the overlap window with `retired:<date>` on the old one
3	Author	Re-sign current bundles with new key; push via `skillctl propose`
4	Reviewer	Re-attest only if policy requires fresh attestation per key (most don't – attestation binds to `bundle_digest`, not signing key)
5	End-User	Pinned `~/.claude/skill-trust-roots.yaml` accepts both keys during overlap. After `retired` date, old-key bundles fail verification on re-install. Existing installs keep working until uninstalled

The same overlap pattern applies to **registry-key rotation** – see `CISO-WORK/RUNBOOK-skill-registry-key-rotation.md`.

4.5 Author-key revocation (SPEC-0198)

When an author key is compromised:

#	Role	Command / action
1	Author (or Operator if author is unreachable)	`POST /api/skills/identities/<id>/revoke` with rationale and effective timestamp
2	Registry	Updates identity row; emits `revoked` graph edges for every bundle signed by the compromised key; publishes `m3c.<env>.skill_identities.revoked` Kafka event
3	KafScale (when active)	Republishes per-tenant; downstream `skillctl install` short-circuits with exit 13/19
4	End-User running `skillctl audit`	Affected skills move to `BROKEN`; `--cleanup` removes them

#	Role	Command / action
5	Auditor	The full chain – fetch sha256, scan report digest, importer signature, reviewer attestation, CISO verdict, revocation row – survives in `_skill_audit_log` for forensic review

4.6 Upstream import path (SPEC-0201)

When a skill comes from a public hub (skillhub.club, claudeskills.info, lobehub.com/skills, etc.), it MUST cross the airlock before reaching the registry:

```
discover  ->  airlock          ->  re-pack/sign  ->  attest      ->  install
hub UI/API    skillctl             skillctl pack    reviewer       skillctl install
              import-public         + sign          attests        (full chain)
```

```
# 1) Discover on a hub UI; copy the slug
skillctl import-public skillhub://didactic-session              # prints required SHA-256 pin,
skillctl import-public skillhub://didactic-session@sha256:abc...  # stages under ~/.cache/m3c/i

# 2) Inspect what landed
skillctl import-list
cat ~/.cache/m3c/imports/<sha256>/scan-report.json

# 3) Promote to a real candidate (existing SPEC-0194 flow)
skillctl propose didactic-session \
    --source ~/.cache/m3c/imports/<sha256>/ \
    --derived-from skillhub://didactic-session@sha256:abc... \
    --intent yellow \
    --rationale "imported from skillhub; pending reviewer"

# 4) Reviewer attests (SPEC-0188 §4.3 -- separate human, not the importer)

# 5) Consumers install (SPEC-0188 §7)
skillctl install didactic-session@1.0.0
```

What the airlock enforces (full list in IMPORT-AIRLOCK.md §"What the airlock enforces"):

1. **Default-deny source policy** (`~/.claude/skill-import-policy.yaml`) – empty file, nothing imports.
2. **Mandatory content pinning** – every fetch is `<scheme>://<ref>@sha256:<hex>`.
3. **Explicit license acceptance** – `--accept-license <SPDX>` or refuse.
4. **Static scanner refuses on:** secret regex hits, `os.system` / shell-true, file writes outside skill dir, HTTP to non-allowlisted hosts, missing license header, CVE-high deps.
5. **Author-intent capped at (Y) yellow.** Imported bundles cannot self-declare green.
6. **Permanent provenance edge** – `(SkillVersion) -[imported_from]-> (ExternalSource host, slug, sha256)`. CISO console renders it as a chip.
7. **Source-level CISO block** – Tenant CISO can block by host pattern (`skillhub.club`); affected installs fail with exit 19.
8. **Forbidden upstream MCP servers** – `@skillhub/mcp-server` is on `mcpServers_blocked`; `skillctl audit` rule R-401 flags any settings.json that re-introduces it.

What NOT to do:

☒ `git clone <hub-repo>` into `~/.claude/skills/` – `skillctl audit` flags `UNVERIFIED`; release-gate refuses.
☒ Add `@skillhub/mcp-server` to `~/.claude/settings.json` – R-401 flags; pre-commit refuses.
☒ Run `import-public` without a pin – the scheme refuses on purpose.
☒ Self-attest (G) on an imported bundle – exit 18; reviewer attestation is mandatory.

---

5. Role playbooks (detailed)

5.1 Author

Daily flow:

1. Edit your skill under `~/.claude/skills/<name>/` or `<repo>/.claude/skills/<name>/`.
2. Update `SKILL.md` frontmatter (`governance_level`, `owner`, `human_checkpoints`, `context_scope` per SPEC-0130 §3).
3. Update `bundle.json` `intent` and `data_dependencies` per SPEC-0196 §3.
4. Bump `version` in `bundle.json` (SPEC-0194 §6 check 10 – semver).
5. `skillctl propose <name> --intent yellow --rationale "<what changed>"` – runs the 10-check ready-to-promote gate (frontmatter complete, governance level set, smoke-test marker present, semver bumped, no open BUG-NNNN against this skill, intent + data_deps declared, etc.). Gate fails loud (exit 2). Fix and re-run.
6. Bundle is admitted; check the queue: `/v2/skills/admin` -> Bundles tab.

You may NOT:

- Bind the governance verdict (Reviewer's job).
- Author skills above the access level of your own user account (SPEC-0130 §6.2).
- Edit `governance_level` post-attestation without re-attestation.
- Author and review the same skill (separation of duties).

Common author errors and fixes:

Symptom	Cause	Fix
`skillctl propose` exits 2 with "frontmatter incomplete"	SKILL.md missing `governance_level` / `owner` / `human_checkpoints`	Add them to frontmatter, re-run
exits 18 `intent_inconsistent`	`intent.network=true` but no HTTP data_dep, OR `intent.destructive=false` but `data_dep[].access=write`	Make `intent` and `data_dependencies` consistent
exits 18 `upstream_intent_capped`	Tried `--intent green` on imported bundle	Use `--intent yellow`; have a reviewer attest (G) separately
keygen produced wrong filename	Output is `<out>.priv` and `<out>.pub`, mode 0600/0644	Move to `~/.claude/skillctl-keys/<name>.{priv,pub}`
Sign produces `<bundle>.<digest_hex>.author.sig` (not just `.author.sig`)	Correct – signature filename embeds digest	Don't rename; verifier looks for the digest-named file

Files you own:

```
~/.claude/skillctl-keys/author.key          (PEM PKCS#8 ed25519, mode 0600)
~/.claude/skillctl-keys/author.pub          (PEM SPKI ed25519, mode 0644)
<repo>/.claude/skills/<name>/               (the skill source)
~/.claude/skills/<name>/                    (your personal skill)
```

5.2 Reviewer

Daily flow:

1. Open `/v2/skills/admin` -> Bundles -> filter “no_governance” or “below_min”.
2. Click the bundle to open its detail panel – see SKILL.md, scripts/, intent, data_dependencies, file CHECKSUMS.
3. Run the **Activation Gate** (SPEC-0130 §5.1):
   - ≥5 test runs (or smoke-test marker)
   - DoD validated against frontmatter
   - `human_checkpoints` defined
   - `context_scope` declared
   - Security review for yellow / red
4. Optional sandboxed dry-install:

```
skillctl install <name>@<version> --home /tmp/review-sandbox-$$ --allow-yellow --verbose
```

5. Issue the verdict:

```
skillctl attest <bundle-digest> \
    --level green \
    --rationale "Activation gate passed; intent matches data_deps; smoke-tested locally." \
    --reviewer-id id:reviewer@m3c \
    --key ~/.claude/skillctl-keys/reviewer.key
```

You may NOT:

- Author bundles you review (separation of duties).
- Bypass the Activation Gate.
- Modify `_skill_bundles`, `_skill_signatures`, `_skill_identities` directly.

v1 limitation: in registry-on-behalf-of mode (CEO briefing Q3 lock) the registry key signs the attestation; this is logged but doesn’t replace per-reviewer keys, which are a Q3-2026 deliverable. Customer engagements should document this in the security narrative.

5.3 CISO

Console layout (`/v2/ciso?tenant=<your-tenant>`):

Tab	What you do
Bundles	Browse all bundles in your tenant scope. Filter by governance, by source, by data dep.
Attestation queue	Process the SLA-tracked queue: **Approve** (attest green tenant-scoped) · **Block** (attest red tenant-scoped -> propagates to consumer-side `skillctl install --tenant <id>` rejection with exit 13) · **Escalate** (queue for next reviewer with SLA timer)
Data sources	Per-(skill, DataSource) chip: `[authorize | deny | pending]`. New (skill, data_source) pairs default to `pending` so you never silently inherit permissions.

Tab	What you do
Runtime (SPEC-0202)	Live tail of active capability tokens. Per row: token id, skill, caller, age, refusal count. `[×]` revoke (≤2s propagation via Kafka). `[a]` attenuate (modal: pick rules to add; child token issued).
Policy	Edit `_skill_runtime_policies` for your tenant: defaults, hard floors, per-skill overrides (see §5.3.2 below).
Audit	Quarterly export PDF/CSV. SPEC-0192 §8 acceptance criterion.

Per-skill review checklist (the panel SPEC-0196 §8 renders):

```
fetch-contract  v1.0.0  (G)
  Trust chain:   author -> registry -> reviewer  [ok]
  Capabilities:  skill:claude-code/scaffolding · skill:document-generation
  Data deps:
    READ:  ds:er1/plm/skill-creations          [authorized]
    WRITE: ds:filesystem/cwd                    [authorized]
    PASS:  ds:http/anthropic-api                [pending]   ← decide
  Side-effects: fs:write, llm:call
  Subprocess:   pandoc, git
  Imported_from: (none)
  Runtime envelope required: true
  [Authorize]  [Deny]  [Quarantine]
```

The chip is the **decision handle**. Authorize by data dependency, not by skill alone. The next install on a consumer machine respects the verdict.

5.3.1 Block-by-source If a hub turns hostile:

```
Settings -> Source policies -> Block source
  host: skillhub.club
  effective: now
  rationale: "Compromised author key reported 2026-05-07."
```

All bundles whose `imported_from.host` matches now fail install with exit 19. Replay-test the verdict via `skillctl invoke-replay` (SPEC-0202).

5.3.2 Runtime policy template `/v2/ciso?tenant=<id>&tab=policy` – edit `_skill_runtime_policies`:

```yaml
defaults:
  max_token_ttl_seconds: 300
  max_runtime_seconds: 120

hard_floors:
  egress_allowlist: [api.anthropic.com, registry.aims-core.example]
  subprocess_denylist: [curl, wget, ssh, rm, kubectl, gcloud]
  destructive_skills_require_dual_token: true

overrides:
  - skill_name: deploy-stage
    max_runtime_seconds: 600
    egress_allowlist_add: [console.cloud.google.com]
```

Any envelope is `(bundle declaration) ∩ (this policy) ∩ (caller request)`. Empty intersection -> no token issued, skill cannot run.

5.3.3 Quarterly audit binder Export from `Audit` tab -> PDF/CSV. Contains:

- Bundle inventory at quarter end
- Every Approve/Block/Escalate verdict by reviewer + timestamp
- Every (skill, data_source) authorization chip change
- Every runtime token issued/revoked/attenuated
- Every `gate.refused` event (envelope violations)
- Every imported bundle and its source chain

This is what auditors and regulators receive. Keep `tenant_scope` on every row – never include other tenants' data.

5.4 Registry Operator

Daily flow is mostly automated. Your scheduled work:

```
cd m3c-tools-maintenance/INFRA/skill-registry/
./bootstrap.sh --tenant <tenant-id> --env <prod|stage>
```

5.4.1 Provisioning a tenant Provisions: GCS bucket `skill-bundles-<tenant>`, Firestore namespace `tenant_<id>`, registry signing key in Secret Manager (`semanpix/skill-registry-key-<tenant>`), default trust-roots template at `tenants/<id>/skill-trust-roots.yaml`.

Distribute `tenants/<id>/skill-trust-roots.yaml` to all consumer machines for that tenant.

```
curl -X POST $REG/api/skills/identities \
    -H "Authorization: Bearer $ADMIN_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{
      "id": "id:author@<org>",
      "ed25519_pubkey": "<base64 SPKI>",
      "tier": "user",
      "tenant_scope": "<tenant-id>",
      "registered_by": "<your-id>",
      "rationale": "<why>"
    }'
```

5.4.2 Registering an authoring identity (v1 manual) v2 (post-2026-Q3) will move this to GitHub OIDC or SSO-backed identity registration.

5.4.3 Key rotation Follow `CISO-WORK/RUNBOOK-skill-registry-key-rotation.md` verbatim. The overlap-publish pattern: `registry_keys[]` carries multiple entries during the window; verifier accepts any non-retired entry. **Abrupt key replacement breaks every consumer's `skill-trust-roots.yaml`** – never skip the overlap window.

```
curl -X POST $REG/api/skills/identities/<id>/revoke \
    -H "Authorization: Bearer $ADMIN_TOKEN" \
    -d '{"effective_at": "<ISO>", "rationale": "<why>"}'
```

5.4.4 Identity revocation Triggers the cascade in §4.5 Author-key revocation.

You may NOT:

- Author or attest skills.
- Make per-tenant policy decisions (CISO's job).
- Modify author signatures or governance attestations directly.

5.5 Tenant Admin

Files in your scope:

```
tenants/<tenant-id>/
├── config.yaml                    # tenant config + SLA tiers + notify-targets
├── skill-trust-roots.yaml         # which registry keys this tenant accepts
├── role-bindings.yaml             # tenant_ciso, tenant_admin, etc.
└── runtime-policy.yaml            # SPEC-0202 envelope policy
```

```
curl -X POST $REG/api/tenants/<tenant-id>/role-bindings \
    -d '{"role": "tenant_ciso", "identity": "id:ciso@<org>"}'
```

5.5.1 Bind the CISO The bound identity gains access to `/v2/ciso?tenant=<id>`.

```
# tenants/<id>/config.yaml
sla:
  P0:
    review_within_hours: 4
    notify: ["#ciso", "ciso@<org>"]
  P1:
    review_within_hours: 24
  P2:
    review_within_hours: 168    # 1 week
notify_targets:
  slack:   "https://hooks.slack.com/services/..."
  email:   "ciso@<org>"
governance_minimum: green
```

5.5.2 Configure SLA tiers Policy attestations below the minimum trigger an SLA-tracked notification (SPEC-0192 §7).

5.5.3 Pause skill fan-out If something is on fire:

```
curl -X POST $REG/api/tenants/<tenant-id>/fanout-pause \
    -d '{"rationale": "incident-2026-05-07-001", "until": "<ISO>"}'
```

KafScale stops republishing to `tenant.<id>.skills.*` until you resume. Existing consumer installs keep working; new admissions wait.

You may NOT: attest, author, rotate keys, edit other tenants' configs.

5.6 End-User / Skill Consumer

```
# 1) Build/install skillctl (your operator distributes a signed binary; verify it)
which skillctl                                                  # /usr/local/bin/skillctl

# 2) Pin trust roots
mkdir -p ~/.config/m3c/skill-trust-roots
# (your operator gives you the registry pubkey; save it locally)
skillctl trust add \
    --registry https://aims.example.com/api/skills \
    --pubkey  ~/.config/m3c/skill-trust-roots/aims-example.pub \
    --id aims-example-prod

skillctl trust list                                             # confirm
```

5.6.1 First-machine setup The trust-roots file is at `~/.claude/skill-trust-roots.yaml`.

```
skillctl install didactic-session@1.0.0 --tenant <your-tenant>
# or, with a yellow override (audited):
skillctl install some-skill@2.0.0 --allow-yellow

# Verify any installed skill (re-runs the chain check)
skillctl verify didactic-session
```

5.6.2 Install a skill Verifier order (SPEC-0188 §7): digest -> author sig -> registry sig -> governance ≥ minimum -> tenant policy -> data_dependencies all authorized -> depends_on resolved -> atomic move to `~/.claude/skills/<name>/`. Any failure = no write. Numbered exit codes §8.

```
skillctl run didactic-session                                   # default envelope
skillctl run didactic-session --tenant <your-tenant>            # tenant-scoped
skillctl run didactic-session --max-runtime 600                 # request larger envelope (interse
```

5.6.3 Run a skill The wrapper:

1. POSTs `/api/skills/capability-tokens/issue` – gets a 5-minute signed token.
2. Sets `M3C_SKILL_CAPABILITY_TOKEN=<token>` in the child env.
3. LD_PRELOAD/DYLD_INSERT_LIBRARIES injects the gateway shim.
4. SOCKS sentinel steers all egress through a tenant proxy that requires the live `token_id` as auth.
5. Skill runs. Every HTTP/subprocess/file call is gated. Refusals -> exit 30-39 + audit row.
6. `invocation.completed` lands on `m3c.<tenant>.skill_invocations`.

You almost never see the envelope – only when you violate it.

```
skillctl audit                                                  # OK / UNVERIFIED / BROKEN / BELOW
skillctl audit --json | jq                                      # for scripts
skillctl audit --cleanup --dry-run-cleanup                      # preview deletions
skillctl audit --cleanup --confirm-delete                       # apply
```

5.6.4 Audit your machine Run weekly (or in pre-commit / CI). `BELOW_MIN` skills are reported but not auto-deleted – you may install with `--allow-yellow` instead.

You may NOT:

- Edit a skill in `~/.claude/skills/` and expect it to keep its trust state – `skillctl audit` flags it `BROKEN`.

- Bypass tenant-scoped block verdicts (exit 13).
- Publish to the registry (Author's job).

5.7 Auditor

Read-only forensics. No write authority anywhere. Independence is the value – never also be Reviewer or CISO for the same scope.

What you read:

Source	What's in it
`_skill_audit_log`	Every state change with actor, timestamp, before/after
`_skill_attestations`	All governance attestations (current + historical)
`_skill_signatures`	Author + registry signatures
`_skill_graph_edges`	Structural + semantic edges
`m3c.<tenant>.skill_invocations` Kafka topic	Every issue, attenuate, revoke, gate.allowed, gate.refused, invocation.completed
`skillctl audit` JSON output across machines	Per-machine truth
`~/.cache/m3c/imports/<sha256>/scan-report.json` (per machine)	Pre-airlock scanner output

Reconstructing a single skill's chain:

```
# Bundle row + signatures
curl $REG/api/skills/by-digest/<digest>
curl $REG/api/skills/<digest>/signatures
curl $REG/api/skills/<digest>/attestations

# Identity provenance
curl $REG/api/skills/identities/<author-id>
curl $REG/api/skills/identities/<author-id>/history       # rotations + revocations

# Audit log chronology
curl $REG/api/skills/audit-log?bundle_digest=<digest>     # ordered events

# Runtime invocations (per tenant)
kafka-console-consumer --topic m3c.<tenant>.skill_invocations --from-beginning \
  | jq 'select(.bundle_digest == "<digest>")'

# Local install state on a machine
ssh <host> 'skillctl audit --json' | jq '.skills[] | select(.bundle_digest == "<digest>")'
```

Issuing findings: you cannot enforce. Findings escalate to:

- CISO (block) for tenant-scoped policy
- Reviewer (re-attest) for governance verdict change
- Registry Operator (revoke identity) for key compromise

5.8 m3c Consultant overlay

Per-role consulting deliverables (from ROLES.md §4):

Role	Consultant deliverable
Author	Author handbook tailored to customer stack + 5 reference skills published in their tenant
Reviewer	Reviewer playbook with N concrete examples graded against customer policy
CISO	Tenant policy rule-set v1 + CISO Console training session + the 7-criterion sign-off
Registry Operator	Tenant bootstrap delivered + key rotation runbook localized + first dry-run rotation completed
Tenant Admin	Tenant config v1 in git + role binding playbook
End-User	Per-user install kit + SKILLOR-WORK-style first-run session log per machine
Auditor	Quarterly audit template + dashboard + reproducible forensics runbook

Exit criterion: customer's roles execute the four interaction patterns (§4) without consultant intervention for one full quarter.

---

6. Decision tree: which command should I run?

```
"I want to..."

  ... see what skills are on this machine        -> skillctl scan
  ... see which are signed/verified/broken       -> skillctl scan --with-trust
  ... compare two scans                           -> skillctl diff
  ... view a scan in browser                      -> skillctl review
  ... open the read-only TUI inventory            -> skillctl browse
  ... freeze current scan as baseline             -> skillctl seal
  ... fix annotation gaps in a baseline           -> skillctl consolidate

  ... pack a skill into a .skb                    -> skillctl pack <skill-dir>
  ... generate a signing keypair                  -> skillctl keygen --out <path>
  ... sign a bundle                          -> skillctl sign <bundle.skb> --key <path>
  ... verify a bundle's author signature locally  -> skillctl verify-sig <bundle.skb> --pubkey <path>
  ... run the ready-to-promote gate + push      -> skillctl propose <name>        [SPEC-0194]

  ... pull metadata of locally-scanned skills
      into the registry without uploading bytes  -> skillctl awareness sync       [SPEC-0195]
  ... read back what was admitted in a session    -> skillctl awareness verify
  ... reset a session's awareness admissions       -> skillctl awareness reset --session <tag>

  ... declare or update a skill's intent +
      data_dependencies (SPEC-0196) without
      touching code                               -> skillctl intent declare <name>

   ... import a skill from a public hub                  -> skillctl import-public <ref>     [SPEC-0201]
```

```
  ... list staged imports                          -> skillctl import-list
  ... lint the import policy                    -> skillctl import-policy lint <path>

  ... pin a registry as a trust root               -> skillctl trust add
  ... list pinned trust roots                      -> skillctl trust list
  ... unpin a registry                             -> skillctl trust remove

  ... install a verified skill                  -> skillctl install <name>@<version>
  ... re-verify an already-installed skill         -> skillctl verify <name>
  ... run an installed skill (envelope-bounded)  -> skillctl run <name>          [SPEC-
0202]
  ... replay an invocation under new policy     -> skillctl invoke-replay <token-id>

  ... sign a governance verdict                     -> skillctl attest <digest> --
level <green|yellow|red>
  ... antivirus-style audit of all installed
      skills                                       -> skillctl audit
  ... clean up broken/unverified installs          -> skillctl audit --cleanup

  ... sync local skill-usage telemetry to
      aims-core                                    -> skillctl sync-usage
```

When in doubt, see the full reference: SKILLCTL-MANUAL.md.

---

7. Maturity model

Per ROLES.md §5 and CONCEPT.md §6.2 – three tiers per role and per process. Maturity refers to **operational practice**, not feature completeness.

Tier	Inventory	Admission	Attestation	Audit	Authorization	Runtime
Bronze	`skillctl scan` produces JSON; no registry	-	-	-	-	-
Silver	Awareness sync to a tenant registry; bundles admitted	Awareness path live	Default attestations only (yellow user-tier, green plugin-tier)	`skillctl audit` weekly per machine	-	-
Gold	`skillctl propose` is daily flow; gate enforces frontmatter discipline	Author path live (full chain)	Named human reviewer attestations; CISO Console operational	Audit verdict in deploy gate (CI/pre-commit)	Per-(skill, data_source) authorization with policy engine	`skillctl run` envelope-bounded; live revoke/attenuate

Tier	Inventory	Admission	Attestation	Audit	Authorization	Runtime
Platinum	Cross-host inventory aggregation; KafScale fan-out	Multi-tenant; KafGraph backend	N-of-M reviewer requirement; per-reviewer keys	Behavioral attestation (sandbox runs catch lying intent)	Cross-skill composition policy (graph walk)	Federated runtime SLAs; signed end-to-end invocation manifests

The maturity tiers double as **billable engagement scopes**:

Phase	Target	Effort	Unlocks
Bronze->Silver	tenant provision	~1-2 weeks	inventory + registry on customer infra
Silver->Gold	CISO process + audit gate	~3-4 weeks	the saleable artifact (CISO sign-off process)
Gold->Platinum	multi-tenant scale-out	ongoing	the federated/regulated tier

---

8. Exit codes

Centralised in `cmd/skillctl/exit.go`. Branching on these is safe; parsing log lines is not.

8.1 Generic

Code	Name	Meaning
0	`OK`	Success
1	`generic`	Other error (read stderr)
2	`usage`	Usage / flag error
3	(reserved – `audit BROKEN`)	At least one skill audited as broken

8.2 SPEC-0188 §11 – bundle / install / verify (10..16)

Code	Name	Meaning
10	`digest_mismatch`	Recomputed digest ≠ bundle name digest
11	`author_sig_invalid`	`<digest>.author.sig` failed verification
12	`registry_not_pinned`	Registry not in `~/.claude/skill-trust-roots.yaml`
13	`governance_below_minimum`	Attested level < trust-root or tenant minimum (use `--allow-yellow` to override per-install, audited)

Code	Name	Meaning
14	`data_dep_denied`	Tenant CISO denied a (skill, data_source) chip
15	`depends_on_unsatisfied`	A `depends_on` skill failed to install
16	`revoked`	Registry attestation says `revoked`

8.3 SPEC-0201 – upstream import airlock (17..19, 4..5)

Code	Name	Meaning	Fix
4	`pin_required`	Fetched without `@sha256:...`	Re-run with the printed pin
5	`scanner_refuse`	Static scan finding above floor	Read `scan-report.json`; either fix upstream or get policy exception
17	`no_source_policy`	Policy file missing/empty/host-not-allowed	Edit `~/.claude/skill-import-policy.yaml`
18	`upstream_intent_capped`	Tried `propose --intent green` on imported bundle	Use `--intent yellow`; reviewer attests (G) separately
19	`source_blocked`	Tenant CISO blocked this source	Talk to the CISO; verdict id in stderr

8.4 SPEC-0195 §11 – awareness / intent (18..19)

Code	Name	Meaning
18	`intent_inconsistent`	`intent` and `data_dependencies` cross-validation failed
19	`identity_mismatch`	Author identity does not match registered identity

(Codes 18/19 are shared between SPEC-0195 and SPEC-0201 by design – both indicate boundary-policy violations at admission time.)

8.5 SPEC-0202 – runtime envelope (30..39)

Code	Name	Meaning
30	`envelope_capability_missing`	Skill tried an op (e.g. `subprocess_run`) not in the envelope
31	`envelope_data_source_missing`	Skill touched a data source not declared / not allowed

Code	Name	Meaning
32	`egress_host_not_allowed`	HTTP request to a host outside `egress_allowlist`
33	`subprocess_denied`	Tried to exec something on `subprocess_denylist`
34	`file_outside_envelope`	Read/write to a path outside declared `data_sources`
35	`token_expired`	TTL elapsed mid-call
36	`token_revoked`	CISO killed the token
37	`token_invalid_signature`	Token bytes tampered
38	`runtime_quota_exceeded`	`max_runtime_seconds` hit
39	`egress_byte_quota_exceeded`	`max_egress_bytes` hit

Plus issuer-side HTTP refusals: `403 governance_below_minimum`, `403 envelope_empty`, `403 source_blocked`, `503 issuer_overloaded`.

---

9. Cookbook

9.1 Onboard a new author on a fresh laptop

```
# 1) Get skillctl
curl -L https://aims.example.com/dist/skillctl-darwin-arm64 -o /usr/local/bin/skillctl
chmod +x /usr/local/bin/skillctl
skillctl --help

# 2) Generate a signing keypair (keep both files local; only .pub leaves the laptop)
mkdir -p ~/.claude/skillctl-keys
skillctl keygen --out ~/.claude/skillctl-keys/author

# 3) Send the .pub to your Registry Operator (Slack/email/issue tracker)
cat ~/.claude/skillctl-keys/author.pub

# 4) Pin the registry trust root they hand back
skillctl trust add --registry https://aims.example.com/api/skills \
    --pubkey ~/.config/m3c/skill-trust-roots/aims-prod.pub \
    --id aims-example-prod

# 5) Verify with a known-good bundle
skillctl install didactic-session@1.0.0 --tenant <your-tenant>
skillctl run didactic-session
```

9.2 First skill ever

```
# Lay it down
mkdir -p ~/.claude/skills/hello-skill
cat > ~/.claude/skills/hello-skill/SKILL.md <<'EOF'
---
name: hello-skill
version: 0.1.0
governance_level: yellow
```

```
owner: id:author@<org>
human_checkpoints: ["confirm-output"]
context_scope: ["fs:write:<cwd>/output/**"]
---

# hello-skill
Writes a hello.txt to ./output/.
EOF
mkdir -p ~/.claude/skills/hello-skill/scripts

# Declare intent + data deps
cat > ~/.claude/skills/hello-skill/bundle.json <<'EOF'
{
  "schema": "m3c-skill-bundle/v1",
  "name": "hello-skill",
  "version": "0.1.0",
  "summary": "Writes a hello.txt.",
  "author_governance_intent": "yellow",
  "author_governance_rationale": "Writes one local file.",
  "intent": {
    "summary": "Writes a hello.txt to ./output/.",
    "side_effects": ["fs:write"],
    "destructive": false,
    "network": false,
    "subprocess": [],
    "claims": []
  },
  "data_dependencies": [
    {"id": "ds:filesystem/cwd", "kind": "local_fs",
     "access": "write", "scope": "<cwd>/output/**",
     "reason": "Write hello.txt."}
  ],
  "runtime_envelope_required": true
}
EOF

# Propose
skillctl propose hello-skill --intent yellow --rationale "First skill."
```

9.3 Triage a “BELOW_MIN” skill across the team

```
# 1) Find affected machines (each person runs):
skillctl audit --json | jq '.skills[] | select(.verdict == "BELOW_MIN")' > /tmp/below_min.json

# 2) Find the cause -- likely a (Y) attestation against a (G)-required tenant
curl $REG/api/skills/by-digest/<digest>/attestations

# 3) Either:
#    a) Reviewer re-attests upward:
skillctl attest <digest> --level green --rationale "<reason>" --reviewer-id <id> --key <path>
#    b) Or: tenant relaxes governance_minimum (CISO Console -> Policy)
#    c) Or: end-users install with --allow-yellow (audited, individual decision)
```

9.4 Investigate a `gate.refused` event

```
# 1) From the CISO Runtime tab, copy the token id and refusal row
TOKEN=ct:01H...mz9
EXIT_CODE=32   # egress_host_not_allowed

# 2) Pull the full invocation chain
kafka-console-consumer --topic m3c.<tenant>.skill_invocations --from-beginning \
  | jq "select(.token_id == \"$TOKEN\")"

# 3) Replay under the same policy to confirm determinism
skillctl invoke-replay $TOKEN --mode dry

# 4) If you want to tighten policy further, replay under the new policy
skillctl invoke-replay $TOKEN --mode live --tenant <tenant>

# 5) If the skill has a legitimate need, edit the runtime policy (CISO):
#    overrides[].egress_allowlist_add += <host>
```

9.5 Quarterly audit binder generation

```
# 1) Trigger the export from the CISO console (Audit tab -> Export quarter)
# 2) Or script it:
curl -X POST $REG/api/tenants/<tenant>/audit/export \
    -d '{"from": "2026-01-01", "to": "2026-03-31", "format": "pdf"}' \
    -H "Authorization: Bearer $TOKEN" \
    --output Q1-2026-skill-audit-<tenant>.pdf

# 3) Cross-check the inventory
skillctl audit --json > Q1-2026-machine-<host>.json   # per machine; collect
```

---

10. Failure modes & remediation

Symptom	Likely cause	Remediation
`firestore.exceptions.PermissionDenied` on awareness sync	Wrong creds chain selected	Set `GOOGLE_APPLICATION_CREDENTIALS_FIRESTOR` to the youtubeanalizer23 service-account key
`skillctl: command not found`	Binary not on PATH	Build via `cd m3c-tools && go build -o /usr/local/bin/skillctl ./cmd/skillctl`; or set `SKILLCTL_BIN`
`skillctl install` exits 12 (`registry_not_pinned`)	The bundle's registry isn't in trust-roots	Operator distributes the registry pubkey; `skillctl trust add`
`skillctl install` exits 13 even after re-attest	Tenant policy still has the old value cached	Bounce the consumer; or `skillctl install --tenant <id>` to force tenant resolution

Symptom	Likely cause	Remediation
Trust Graph empty in admin UI even though Bundles tab shows entries	Graph-edge writer didn't run (BUG-0131 territory)	Check container logs for `graph-edge-failed` warnings; if it recurs, file BUG-NNNN with the warning text
`skillctl pack` exits 18 `intent_inconsistent`	Cross-validation found `intent.network=false` but a `data_dep` declares an HTTP source (or similar)	Make `intent` and `data_dependencies` consistent and re-run
`skillctl run` exits 32 (`egress_host_not_allowed`)	Skill tried to reach a host not in tenant `egress_allowlist`	Either the skill is buggy (fix it), or CISO needs to add the host to the tenant policy
Skill counted in inventory but no bundle in admin UI after awareness sync	Ingest skipped due to digest collision (existing different-name doc)	Re-run with `--help-advanced` to see `--allow-overwrite` (logs collisions but proceeds)
Container shows "0 bundles" after run	Firestore project mismatch	Confirm container env `FIRESTORE_PROJECT_ID=youtubeanalizer23`; recompose with `/dev-cycle`
`skillctl run` hangs / SOCKS sentinel rejects	Token expired (default 5 min)	Re-issue: `skillctl run` will mint a new one. For long-running, request `--max-runtime <N>`
Pre-commit hook fails on `~/.claude/skills/` edit	`skillctl audit` flags `BROKEN` (file modified post-install)	Either revert the edit, or re-author through `skillctl propose` (don't mutate installed bundles)

---

11. References

- **Project docs:** README.md · CONCEPT.md · ROLES.md · GAP-ANALYSIS.md
- **Operator briefs:** IMPORT-AIRLOCK.md (SPEC-0201) · RUNTIME-ENVELOPE.md (SPEC-0202)
- **Live procedure:** SKILLOR-WORK/s1/USER-MANUAL.md
- **CLI reference:** SKILLCTL-MANUAL.md
- **SPECs:** -0114 · -0115 · -0121 · -0122 · -0130 · -0188 · -0189 · -0190 · -0192 · -0193 · -0194 · -0195 · -0196 · -0197 · -0198 · -0200 · -0201 · -0202
- **Governance handbook:** `.claude/GOVERNANCE.md` (active SPEC-0130)
- **Field bugs (the friction that shaped the design):** BUG-0131 · BUG-0132 · BUG-0133

skillctl – Command Reference Manual

This is the comprehensive command reference for `skillctl`, the Go CLI front-end for the m3c Skill-Manager. It extends `SKILLOR-WORK/s1/USER-MANUAL.md` (which covered only the awareness-sync subset) with the full surface area: every subcommand from the `s2/integration` branch's `cmd/skillctl/main.go` dispatcher, plus the SPEC-0194/-0196/-0201/-0202 commands that are drafted but not all wired yet.

For role-keyed playbooks (when to run what, in what order, as which role), use USER-MANUAL.md.

Conventions

Notation	Meaning
`<arg>`	Required positional argument
`[--flag VALUE]`	Optional flag
`--flag VALUE | OTHER`	Pick one of the values
`[shipped]`	Wired in `s2/integration`, runnable today
`[drafted]`	SPEC published, code in flight or staged
`[planned]`	Interface frozen in SPEC, no code yet

Status legend:

- (G) = green-track – `skillctl pack`, `keygen`, `sign`, `verify-sig`, `attest`, `trust`, `install`, `verify`, `scan`, `report`, `diff`, `seal`, `import`, `audit`, `review`, `browse`, `consolidate`, `sync-usage`, `intent`, `awareness sync|verify|reset`
- (Y) = drafted – `propose` (SPEC-0194), `import-public/import-list/import-policy lint` (SPEC-0201), `run/invoke-replay` (SPEC-0202)
- (R) = planned – KafScale-coupled fan-out commands (SPEC-0193)

The dispatcher's hardcoded set today: `pack | keygen | sign | verify-sig | trust | attest | install | verify | scan | report | diff | seal | import | audit | review | browse | consolidate | sync-usage | intent | awareness | help`. Drafted commands listed below as `[drafted]` are documented at the planned interface; verify with `skillctl <cmd> --help` before scripting against them.

Default key/file locations:

Path	Contents	Mode
`~/.claude/skillctl-keys/author.{priv,pub}`	Author signing keypair (PEM PKCS#8 / SPKI ed25519)	0600 / 0644
`~/.claude/skillctl-keys/reviewer.{priv,pub}`	Reviewer signing keypair (post-Q3-2026 personal-key path)	0600 / 0644
`~/.claude/skill-trust-roots.yaml`	Pinned registry public keys + governance minima + tenant scope	0644
`~/.claude/skill-import-policy.yaml`	SPEC-0201 source policy	0644
`~/.cache/m3c/imports/<sha256>/`	Staged upstream imports (post-airlock, pre-propose)	0700
`~/.config/m3c/skill-keys/`	Suggested location for `keygen --out`	0700
`~/.m3c-tools/skill-usage.db`	SQLite skill-usage telemetry (synced via `sync-usage`)	0600

Environment variables:

Variable	What	Used by
`M3C_REGISTRY_URL`	Default registry base URL (override per-command via `--registry`)	awareness, attest, install, verify, etc.
`SKILLCTL_BIN`	Path to a non-PATH skillctl binary	wrappers (e.g. `tools.skill_awareness.run`)

Variable	What	Used by
`SKILLCTL_DEV_SEED`	Use deterministic dev keypair seeded from this string (DEV ONLY)	awareness sync (test/dev)
`M3C_SKILL_CAPABILITY_TOKEN`	Live capability token (set by `skillctl run`; consumed by gateway shim)	runtime envelope (SPEC-0202)
`GOOGLE_APPLICATION_CREDENTIALS_FIRESTORE`	Firestore creds path	server-side flows

Exit codes: documented per-command below; the centralised catalog is in USER-MANUAL.md §8.

Top-level usage

```
Usage: skillctl <command> [args]

Commands (Stream S1 / SPEC-0188 Phase 2):
  keygen       Generate an ed25519 keypair (PEM, PKCS#8 / SPKI).
  sign         Sign a .skb bundle with an ed25519 private key.
  verify-sig   Verify a detached author signature locally.
  attest       POST a signed governance attestation to the registry.

Commands (Stream S7 / SPEC-0188 Phase 4):
  trust        Manage ~/.claude/skill-trust-roots.yaml (list/add/remove).

Commands (Stream S8 / SPEC-0188 Phase 4):
  install      Pull, verify, and install a signed skill bundle.
  verify       Re-run the trust-chain check on an installed skill.

Commands (SPEC-0188 Phase 1 PoC):
  pack         Build a deterministic .skb tar from a skill directory.

Commands (SPEC-0189 / inventory):
  scan         Walk skill directories; emit JSON/table inventory.
  report       Render a scan as HTML or markdown.
  diff         Compare two scans.
  seal         Freeze a baseline scan.
  import       Push a scan to a remote target (--dry-run available).
  audit        Antivirus-shaped scanner (verdict per skill).
  review       Open a local UI to review a delta.
  browse       Read-only TUI inventory browser.
  consolidate  Fix annotation gaps in a baseline.
  sync-usage   Sync local skill-usage telemetry.

Commands (SPEC-0195 / awareness bridge):
  awareness sync     Admit local skills to a registry.
  awareness verify   Read back per-session admissions.
  awareness reset    Delete a session's admissions (with dry-run-token).

Commands (SPEC-0196 / intent):
  intent declare    Declare or update intent + data_dependencies for a bundle.
```

```
Commands (SPEC-0194 / author ingestion) [drafted]:
  propose      Run ready-to-promote gate, then pack + sign + push.

Commands (SPEC-0201 / upstream airlock) [drafted]:
  import-public        Stage a hub bundle under ~/.cache/m3c/imports/<sha256>/.
  import-list          List staged imports.
  import-policy lint   Validate ~/.claude/skill-import-policy.yaml.

Commands (SPEC-0202 / runtime envelope) [drafted]:
  run             Issue a capability token, wrap invocation, gate side-effects.
  invoke-replay   Re-run a recorded invocation under old or new policy.

Run any command with --help for its flags.
```

Authoring (signing, packing, verifying)

`pack` – build a deterministic `.skb` (G)

Purpose. Convert a skill directory into a deterministic gzip-tar `.skb` whose SHA-256 is the canonical `bundle_digest`.

Usage.

```
skillctl pack <skill-dir> [--out <path>] [--name <name>] [--version <semver>]
```

Arguments:

- `<skill-dir>` – directory containing `SKILL.md`, `bundle.json`, optional `scripts/`, `helpers/`, `profiles/`, etc.
- `--out PATH` – output `.skb` (default: `<name>-<version>.skb` in the current directory).
- `--name NAME` – override `bundle.json::name` (rare).
- `--version SEMVER` – override `bundle.json::version` (rare).

Determinism rules (SPEC-0188 §3.1):

- Files in lexicographic order
- Mode `0644` (or `0755` under `scripts/`)
- mtime = Unix epoch (`1970-01-01T00:00:00Z`)
- Tar PAX format with no extended headers
- Gzip header zeroed (`gzip --no-name` semantics)
- No `.DS_Store` / `.git` at root
- No wrapper directory (files at tar root, per BUG-0133 resolution)

Cross-field validation (SPEC-0196 §3.3) – `pack` refuses with `exit 18 intent_inconsistent` when:

- `intent.destructive=true` and any `data_dependency.access` is `read-only`
- `intent.network=true` and no `data_dependency.kind=http_endpoint`
- `data_dependency.access=write` and `intent.destructive=false`
- `intent` is missing AND `data_dependencies` is non-empty (or vice versa)

Output.

```
wrote bundle.skb (sha256:cd1d5c79...)
digest: sha256:cd1d5c79...
files:  12
bytes:  20480
```

Exit codes: `0` ok · 1 generic · 2 usage · 18 intent_inconsistent

Gotchas.

- The bundle's filename embeds neither the digest nor a wrapper directory by default. Two `pack` runs produce identical bytes (you can `cmp` them).
- `built_at` in `bundle.json` is *always* Unix epoch in the packed bundle. The wall-clock build time lives in the registry attestation.

---

keygen – generate an ed25519 keypair (G)

Purpose. Generate a fresh ed25519 keypair for use with `sign`, `attest`, `awareness sync`, etc.

Usage.

```
skillctl keygen --out <path>
```

Output files.

```
<path>.priv     PEM PKCS#8, ed25519 (mode 0600)
<path>.pub      PEM SPKI,   ed25519 (mode 0644)
```

Suggested location. `~/.config/m3c/skill-keys/<name>` or `~/.claude/skillctl-keys/<role>` (the latter is what the rest of the CLI defaults to).

Stdout.

```
wrote <path>.priv (mode 0600)
wrote <path>.pub  (mode 0644)
fingerprint: sha256:<hex>
```

Exit codes: `0` ok · 1 generic · 2 usage

Gotchas.

- Keep the `.priv` file off git. The `gitleaks` config from `/security-scan` flags `*.priv`; pre-commit hooks should refuse it.
- For dev/test, use `SKILLCTL_DEV_SEED=<string>` instead – produces deterministic keys for repeatable tests (NEVER in prod).

`sign` – sign a `.skb` bundle (G)

Purpose. Compute the bundle's SHA-256 digest, sign the 32 raw bytes with an ed25519 private key, write a detached signature file.

Usage.

```
skillctl sign <BUNDLE.skb> --key <PATH.priv> [--identity-id <ID>]
```

Output file.

```
<BUNDLE.skb>.<digest_hex>.author.sig    (64 raw bytes, mode 0644)
```

The signature filename embeds the digest so multiple bundles sharing a directory don't collide.

Stdout.

```
digest: sha256:<hex>
identity: <ID>
fingerprint: sha256:<key-fingerprint-hex>
```

`sig: <BUNDLE.skb>.<digest_hex>.author.sig`

Flags.

- `--key PATH.priv` (required) – PEM PKCS#8 ed25519 private key
- `--identity-id ID` – override the author identity (default: `id:<user>@m3c`)

Exit codes: `0` ok · 1 generic · 2 usage · 11 author_sig_invalid (rare; only on internal verify-after-sign self-check)

`verify-sig` – verify a detached author signature locally (G)

Purpose. Recompute the bundle's SHA-256 digest, locate the matching `<BUNDLE.skb>.<digest_hex>.author.sig`, verify it against a public key. **Local check only; does not consult the registry.** Use `verify` (no `-sig` suffix) for the full chain.

Usage.

```
skillctl verify-sig <BUNDLE.skb> --pubkey <PATH.pub>
```

Output (success).

```
OK: signature verified
```

Exit codes: `0` ok · 11 signature invalid · 1 other error · 2 usage

---

Trust-root management

`trust list` – show pinned registries (G)

Purpose. Print the parsed contents of `~/.claude/skill-trust-roots.yaml`.

Usage.

```
skillctl trust list
```

Sample output.

```
registries:
  - id: aims-example-prod
    url: https://aims.example.com/api/skills
    pubkey_sha256: <hex>
    governance_minimum: green
    tenant_scope: kup-berlin
```

Exit codes: `0` ok · 2 usage

`trust add` – pin a registry (G)

Purpose. Add a registry to the trust-roots file.

Usage.

```
skillctl trust add --registry <url> --pubkey <path> [--id <key-id>]
```

Flags.

- `--registry URL` (required) – registry base URL (e.g. `https://aims.example.com/api/skills`)
- `--pubkey PATH` (required) – PEM SPKI ed25519 public key file
- `--id KEY-ID` – optional label (defaults to a short fingerprint-derived id)

Effect. Appends a `registries[]` entry. Idempotent on `(url, pubkey_sha256)`.

Exit codes: `0` ok · 1 generic · 2 usage

`trust remove` – unpin a registry (G)

```
skillctl trust remove --registry <url>
```

Removes the matching `registries[]` entry. After this, `install`/`verify` of any bundle whose registry was pinned at this URL will exit `12 registry_not_pinned`.

Exit codes: `0` ok · 1 generic · 2 usage

Inventory

`scan` – walk skill directories (G)

Purpose. Discover all skills on the machine, classify by tier (project · user · plugin), report shadowing, optionally cross-reference with sibling `.skb` bundles for trust labels.

Usage.

```
skillctl scan [--source claude|user|plugins|all] [--format json|table] [--with-trust]
              [--root <path>] [--out <file>]
```

Flags.

- `--source` – which tier(s) to walk (default `claude`, which is `project + user + plugins` resolved as Claude Code does)
- `--format` – `table` (default for human terminals) or `json` (for scripts)
- `--with-trust` – for each skill, look for sibling `<bundle>.skb` and `<digest>.author.sig`; recompute digest and label `verified | unverified | broken`
- `--root PATH` – alternate scan root (useful for tests)
- `--out FILE` – write to file instead of stdout

Output (JSON).

```
{
  "schema": "m3c-skill-inventory/v1",
  "scanned_at": "2026-05-07T...",
  "host": "<hostname>",
  "total_count": 50,
  "by_tier": {"project": 0, "user": 14, "plugin": 36},
  "by_project": {"<repo>": 0, "<user-home>": 50},
  "skills": [
    {
      "name": "didactic-session",
      "version": "1.0.0",
      "tier": "user",
      "skill_md_sha256": "...",
```

```
      "trust": "verified",            // when --with-trust
      "bundle_digest": "sha256:...",
      "shadowed_by": null,
      "frontmatter": {...},
      "intent_status": "declared"     // or "missing", "unknown" (SPEC-0196 §7)
    }
  ]
}
```

Exit codes: `0` ok · 1 generic · 2 usage

Anchored on `SKILL.md` (not loose `.md`). A directory without `SKILL.md` is not a skill, even if it lives under `~/.claude/skills/`.

`report` – render a scan as HTML or markdown (G)

`skillctl report [--format html|md] --input <scan.json> [--out <file>]`

Useful for sharing a scan with someone who doesn't have skillctl. HTML is self-contained; markdown is GitHub-flavored.

Exit codes: `0` ok · 2 usage

`diff` – compare two scans (G)

`skillctl diff <scan1.json> <scan2.json> [--output json|md|html|text] [--out <file>]`

Reports added / removed / version-changed / digest-changed / trust-state-changed skills. Used to track drift across days or across machines.

Exit codes: `0` ok · 2 usage · non-zero if differences found (script-friendly; check stdout for the verdict)

`seal` – freeze a scan as baseline (G)

`skillctl seal [--input <scan.json>] [--out <baseline.json>]`

Writes a sealed-baseline file you can `diff` against later. `seal` runs over the most recent in-memory scan if `--input` omitted.

Exit codes: `0` ok · 1 generic · 2 usage

`import` – push a scan to a remote target (G)

`skillctl import --target <url> [--api-key <key>] [--input <scan.json>] [--dry-run]`

Generic upload for scan JSON. Distinct from `awareness sync` (which is the trust-aware admission path) and from the SPEC-0201 `import-public` (which fetches from upstream hubs).

Exit codes: `0` ok · 1 generic · 2 usage

`audit` – antivirus-shaped scanner (G) (Phase 2 implementation underway)

Purpose. Different command from `scan`, different default UX, different exit codes. Walks the same three tiers and labels each skill: `OK | UNVERIFIED | BROKEN | BELOW_MIN`.

Usage (planned interface – current build prints "audit: not yet implemented (Phase 2)" for the 1-arg form).

```
skillctl audit [--source claude|user|plugins|all] [--minimum-governance green|yellow]
               [--json] [--cleanup [--dry-run-cleanup | --confirm-delete]]
               [--allow-yellow]
```

Verdicts.

- `OK` – chain validates against pinned trust roots; governance ≥ minimum
- `UNVERIFIED` – no sibling `.skb` / no signature available
- `BROKEN` – chain fails (digest mismatch, sig invalid, registry not pinned)
- `BELOW_MIN` – chain validates but governance below minimum

Cleanup.

- `--cleanup --dry-run-cleanup` – preview which directories would be deleted
- `--cleanup --confirm-delete` – apply (deletes `BROKEN` and `UNVERIFIED`; `BELOW_MIN` reported but not auto-deleted – user may install with `--allow-yellow` instead)

Output footer: `47 OK · 2 unverified · 1 broken · 0 below-min`.

Exit codes: `0` all OK · 2 usage · 3 at least one BROKEN · re-runs are cheap (cached by `skill_md_sha256`)

Use in CI / pre-commit:

```
skillctl audit --json --minimum-governance green || exit 1
```

`review` – open the local TUI for a delta (G)

```
skillctl review [--input <delta.json>] [--port 9115] [--no-browser]
```

Spawns a local browser-based viewer at `http://127.0.0.1:9115/` showing the `diff` output with annotation hooks. `--no-browser` for headless runs.

Exit codes: `0` ok · 2 usage

`browse` – read-only inventory TUI (G)

```
skillctl browse [--scan <scan.json>]
```

In-terminal browser for the inventory: list view, detail view, search. Read-only.

`consolidate` – fix annotation gaps in a baseline (G)

```
skillctl consolidate [--baseline <baseline.json>] [--fix]
```

Without `--fix`: reports gaps. With `--fix`: prompts and applies fixes interactively.

`sync-usage` – push local skill-usage telemetry (G)

Purpose. Read unsynced rows from the local SQLite database (`~/.m3c-tools/skill-usage.db`) and `POST` each to `aims-core /api/v2/skills/usage`. Successfully synced rows are marked `synced=1`.

Usage.

```
skillctl sync-usage --target <url> [--api-key <key>]
```

Exit codes: `0` ok · 1 generic · 2 usage

Why this exists. Skill usage events are recorded locally first (offline-friendly) and pushed in batches; survives outages without losing telemetry.

Awareness bridge (SPEC-0195)

`awareness sync` – admit local skills to a registry (G)

Purpose. Productized version of the SKILLOR-WORK/s1 procedure. Take a `skillctl scan` inventory and admit one bundle per scanned skill via `POST /api/skills/admit-from-scan`. Bundles get `gcs_uri = local-aware://<digest>` – registry knows they exist; bytes are not transferable until republished via the Author path.

Usage.

```
skillctl awareness sync [flags]
```

Flags.

- `--source claude|user|plugins|all` – scan source (ignored if `--inventory` is set; default `claude`)
- `--inventory FILE` – read scan JSON from FILE; pass – to read from stdin
- `--registry URL` – registry URL (overrides trust-roots default_registry and `$M3C_REGISTRY_URL`)
- `--default-attest yellow|green|none` – after admission, request a default attestation (default `none`)
- `--default-intent yellow|green` – stamp this governance level on entries with no/UNKNOWN intent (empty = off)
- `--require-intent` – refuse to send entries with no intent or the SPEC-0196 UNKNOWN sentinel
- `--session TAG` – session tag (default `skill-awareness/<host>/<YYYY-MM-DD>`)
- `--dry-run` – build the envelope; do not POST
- `--confirm` – required to actually push (defense against accidental writes)
- `--key PATH` – author key path (PEM PKCS#8). Default `~/.claude/skillctl-keys/author.key`
- `--author-identity ID` – override the author identity. Default `id:<user>@m3c` (or DevSeedSentinel when `SKILLCTL_DEV_SEED` is set)
- `--help-advanced` – print advanced flags (e.g. `--allow-overwrite` footgun-resistance)

Output.

```
== skill-awareness DRY RUN ==
Backend: <registry url>
Inventory: 50 skills (36 plugin, 14 user)
Reset plan: ...
Ingest plan: ...
NO WRITES PERFORMED. Re-run without --dry-run to execute.
```

Idempotent on `(digest, session_tag)`. Re-running with the same inventory upserts (same digest -> same document path).

Exit codes: 0 ok · 1 generic · 2 usage · 11 author_sig_invalid · 18 intent_inconsistent · 19 identity_mismatch

See also. SKILLOR-WORK/s1/USER-MANUAL.md – the original session that drove SPEC-0195's design.

`awareness verify` – read back per-session admissions (G)

```
skillctl awareness verify [--session TAG] [--registry URL]
```

Reads back what was admitted under a session tag. Useful for confirming the round-trip after a sync. Default session matches the same auto-generated tag as `sync` (`skill-awareness/<host>/<YYYY-MM-DD>`).

Output. Table of `(digest, name, version, attest_level, admitted_at)`.

Exit codes: 0 ok · 2 usage · 1 generic

`awareness reset` – delete a session's admissions (G)

Purpose. DESTRUCTIVE. Delete only the registry docs tagged with the given session – never global, never cross-session. Two-step: dry-run produces a 5-min token; confirm-reset uses that token.

Usage.

```
# Step 1 -- dry run; produces a token
skillctl awareness reset --session TAG --dry-run

# Step 2 -- apply (within 5 min)
skillctl awareness reset --session TAG --confirm-reset --dry-run-token <sig>
```

Flags.

- `--registry URL` – registry base URL
- `--session TAG` (required) – the session whose docs to delete
- `--dry-run` – preview affected docs and produce the 5-minute token
- `--dry-run-token TOKEN` – the token from a prior `--dry-run`
- `--confirm-reset` – required (alongside `--dry-run-token`) to actually DELETE

Why two-step? The destructive blast radius is narrow by design. The dry-run token forces an explicit confirmation path; expired tokens force a re-preview.

Exit codes: 0 ok · 1 generic · 2 usage · 13 token expired (after 5 min)

Intent declaration (SPEC-0196)

`intent declare` – declare or update intent + data_dependencies (G)

Purpose. Update a bundle's `intent` and `data_dependencies` declarations in the registry without re-packing. Useful for filling in missing-intent (UNKNOWN) sentinels left by `awareness sync`.

Usage.

```
skillctl intent declare <skill-name|@digest> [flags]
```

Flags (illustrative – run `skillctl intent declare --help` for the canonical list):

- `--side-effect TOKEN` (repeatable) – `fs:write`, `fs:read`, `network:outbound`, `subprocess`, `secrets:read`, `llm:call`, …
- `--destructive true|false`
- `--network true|false`
- `--subprocess BINARY` (repeatable)
- `--claim CAPABILITY` (repeatable) – `skill:document-generation`, etc.
- `--data-dep ID:KIND:ACCESS:SCOPE:PAYLOAD_CLASS:RETENTION` (repeatable)
- `--reason FREE-TEXT` (required)
- `--key PATH.priv` (required)
- `--author-identity ID`

Cross-validation. Same rules as `pack` (above). Refuses with `exit 18 intent_inconsistent` on contradiction.

Exit codes: `0` ok · 1 generic · 2 usage · 18 intent_inconsistent · 19 identity_mismatch

---

Author ingestion (SPEC-0194)

`propose` – ready-to-promote gate + pack + sign + push (Y) [drafted]

Purpose. The author's daily flow. Runs the SPEC-0194 §6 ten-check ready-to-promote gate, then `pack` -> `sign` -> `POST /api/skills/bundles`.

Usage (planned).

```
skillctl propose <skill-name> [flags]
```

Flags.

- `--source PATH` – explicit source dir (default: resolves `<skill-name>` via project + user + plugins precedence)
- `--intent yellow|green|red` (required) – author's advisory governance intent
- `--rationale TEXT` (required) – what changed, why
- `--key PATH.priv` – author key (default `~/.claude/skillctl-keys/author.key`)
- `--registry URL` – overrides trust-roots default
- `--derived-from REF` – if this skill was imported via SPEC-0201, the upstream ref
- `--dry-run` – run the gate; do not pack or push

The 10 checks (SPEC-0194 §6):

1. Frontmatter complete (`name`, `version`, `governance_level`, `owner`, `human_checkpoints`, `context_scope`)
2. `governance_level` set
3. Smoke-test marker present (or `--allow-no-smoke` with rationale)
4. Semver bumped vs last admitted version
5. No open BUG-NNNN against this skill
6. `intent` declared in `bundle.json`
7. `data_dependencies` declared in `bundle.json`
8. `intent`/`data_dependencies` cross-validation passes
9. `runtime_envelope_required` set (default `true` for new bundles per SPEC-0202)
10. License header present (SPDX) when `imported_from` is set

Exit codes: `0` ok · 1 generic · 2 usage · 5 scanner_refuse · 18 intent_inconsistent / upstream_intent_capped · 19 identity_mismatch

Gate fails loud. No silent skips. Fix and re-run.

Verifier (SPEC-0188 §7, §11)

`install` – pull, verify, install a signed skill bundle (G)

Purpose. The end-user's primary command. Pulls a bundle by name@version (or digest), runs the full chain check, resolves `depends_on`, atomically extracts to `~/.claude/skills/<name>/`. **Any failure = no write.**

Usage.

```
skillctl install <name>[@<version>] [flags]
```

Flags.

- `--registry URL` – registry base URL (required only when trust-roots has multiple registries pinned)
- `--governance-min green|yellow` – override the trust-root's `governance_minimum` (empty = use trust-root)
- `--allow-yellow` – lower the gate from green to yellow for this install (audited)
- `--ignore-deps` – skip `depends_on` resolution (audited)
- `--tenant ID` – pin this install to a tenant scope (SPEC-0188 §7 step 5.5). Overrides trust-roots `tenant_scope`. Empty = use trust-roots value or run untenanted.
- `--verbose` – print structured per-step log lines to stderr
- `--home PATH` – override the install root (advanced; defaults to `$HOME`)

Chain check order (SPEC-0188 §7):

1. Resolve digest via `/api/skills/by-name`
2. Recompute digest from fetched bytes; compare -> exit 10 on mismatch
3. Verify `<digest>.author.sig` against the registered author identity -> exit 11 on fail
4. Verify `<digest>.registry.sig` against pinned trust roots -> exit 12 on fail
5. Read `<digest>.governance.sig`; check governance level ≥ minimum -> exit 13 on fail 5.5. Apply tenant policy -> exit 13 (block) or exit 16 (revoked) as applicable
6. Check all `data_dependencies` are `authorized` for the tenant -> exit 14 on any `denied`
7. Resolve `depends_on` recursively -> exit 15 on any unsatisfied
8. Atomic move to `~/.claude/skills/<name>/`

Exit codes: `0` ok · 1 generic · 2 usage · `10..16` per chain check (see USER-MANUAL.md §8.2)

Reverse the install. Either `rm -rf ~/.claude/skills/<name>/` (then `skillctl audit` will not list it) or use the planned `skillctl uninstall` (queued).

`verify` – re-run the trust-chain check on an installed skill (G)

Purpose. Same chain check as `install`, but against an already-installed skill (no re-fetch). Use after `audit` finds something suspicious.

Usage.

```
skillctl verify <name> [flags]
```

Flags.

- `--registry URL` – same as `install`
- `--governance-min green|yellow`

- `--allow-yellow` – permit yellow result against a green-required trust root (does NOT re-audit; per-call)
- `--tenant ID` – pin this verify to a tenant scope
- `--verbose`
- `--home PATH`

Exit codes: identical to `install` (`0`, 2, `10..16`).

Governance attestation

`attest` – POST a signed governance attestation (G)

Purpose. Reviewer signs `(bundle_digest, governance_level, review_timestamp, reviewer_identity)` and posts to `_skill_attestations`. Multiple attestations per bundle allowed; the **most recent** by a reviewer with the required role is the binding verdict.

Usage.

```
skillctl attest <bundle-digest> \
    --level green|yellow|red \
    --rationale TEXT \
    --reviewer-id ID \
    --key PATH.priv \
    [--registry URL]
```

Flags.

- `<bundle-digest>` (positional, required) – `sha256:<hex>`
- `--level green|yellow|red` (required) – governance verdict
- `--rationale TEXT` (required) – free-text rationale (audit metadata; **NOT** folded into signed bytes)
- `--reviewer-id ID` (required) – reviewer identity (e.g. `id:reviewer@m3c`)
- `--key PATH.priv` (required) – PEM PKCS#8 ed25519 private key (mode 0600)
- `--registry URL` – registry base URL (default: `https://aims-core/api/skills` or env)

v1 limitation. In registry-on-behalf-of mode (CEO briefing Q3 lock) the registry key signs the attestation; the audit log records the human reviewer, but the byte-level signature is the registry's. Per-reviewer keys land Q3-2026.

Exit codes: `0` ok · 1 generic · 2 usage · 11 author_sig_invalid (the signed payload couldn't be re-verified server-side) · 19 identity_mismatch (reviewer-id not registered)

Upstream airlock (SPEC-0201)

`import-public` – stage an upstream bundle (Y) [drafted]

Purpose. Fetch a skill from a public hub, place it under `~/.cache/m3c/imports/<sha256>/`, run static scanners, write `scan-report.json`. **Does NOT install or admit** – staging only.

Usage.

```
skillctl import-public <scheme>://<ref>[@sha256:<hex>] [flags]
```

Schemes. `skillhub`, `claudeskills`, `lobehub`, `skillforge`, `tonsofskills`, `skillsmp`, `gh` (GitHub) – gated by `~/.claude/skill-import-policy.yaml`.

Two-step pinning. First call without `@sha256:...` prints the required pin and refuses (exit 4). Second call with the pin stages the bytes.

Flags.

- `--accept-license SPDX` (required) – explicit license acknowledgement
- `--dry-run` – fetch + scan; do not stage
- `--policy PATH` – override `~/.claude/skill-import-policy.yaml`

Refuses on: secret regex hits, `os.system` / shell-true, file writes outside skill dir, HTTP to non-allowlisted hosts, missing license header, CVE-high deps.

Exit codes: `0` ok · 2 usage · 4 pin_required · 5 scanner_refuse · 17 no_source_policy · 19 source_blocked

---

`import-list` – list staged imports (Y) [drafted]

```
skillctl import-list [--json]
```

Lists everything under `~/.cache/m3c/imports/`. Per row: source ref, sha256, license, scanner verdict, age.

`import-policy lint` – validate the source policy (Y) [drafted]

```
skillctl import-policy lint <path>
```

Validates `~/.claude/skill-import-policy.yaml` against the SPEC-0201 schema. Reports unknown keys, missing required fields, conflicting `allow`/`deny` rules.

Exit codes: `0` valid · 1 invalid (printed) · 2 usage

Runtime envelope (SPEC-0202)

`run` – wrap an invocation under a capability token (Y) [drafted]

Purpose. Issue a short-lived signed capability token, set `M3C_SKILL_CAPABILITY_TOKEN` in the child env, inject the gateway shim via LD_PRELOAD/DYLD_INSERT_LIBRARIES, exec the skill. Every HTTP/subprocess/file call is gated; refusals are audited.

Usage.

```
skillctl run <name> [flags]
```

Flags.

- `--tenant ID` – tenant scope (intersected with caller policy)
- `--max-runtime SECONDS` – request a larger envelope (intersected by tenant policy)
- `--max-egress-bytes N`
- `--capability TOKEN` (repeatable) – request additional capabilities (e.g. `subprocess_run:bash`); intersected with bundle declaration and tenant policy
- `--dry-issue` – issue the token; print the envelope; do not exec

Envelope = `(bundle declaration) ∩ (tenant policy) ∩ (caller request)`. Empty intersection -> no token issued, exit 1 with `403 envelope_empty`.

Defenses against bypass (SPEC-0202):

1. **Bundle declaration** – `runtime_envelope_required: true` (default for new bundles); reviewer attesting (G) verifies the skill imports the gateway; `skillctl pack` lints it.
2. **`skillctl run` wrapper** – LD_PRELOAD/DYLD_INSERT_LIBRARIES injects the gateway shim into the process tree.
3. **SOCKS egress sentinel** – per-tenant outbound network steered through a SOCKS proxy that requires the active `token_id` as auth.

A skill cannot escape all three without the consumer machine being fundamentally compromised.

Exit codes: runtime gate violations surface verbatim – `30` envelope_capability_missing · `31` envelope_data_source_missing · `32` egress_host_not_allowed · `33` subprocess_denied · `34` file_outside_envelope · `35` token_expired · `36` token_revoked · `37` token_invalid_signature · `38` runtime_quota_exceeded · `39` egress_byte_quota_exceeded · plus `403 governance_below_minimum`, `403 envelope_empty`, `403 source_blocked`, `503 issuer_overloaded` from the issuer.

`invoke-replay` – re-run a recorded invocation (Y) [drafted]

Purpose. The Kafka event log makes invocations deterministic. Replay against either the recorded policy (`--mode dry`, sanity check) or a new tighter policy (`--mode live`, pen-test).

Usage.

```
skillctl invoke-replay <token-id> --mode dry|live [--tenant ID] [--policy PATH]
```

Use cases.

- Validate that a policy change actually closes a class of behavior.
- Demonstrate to an auditor that a past invocation was within envelope.
- Pen-test a new policy against historical traffic before publishing.

Output. Sequence of `gate.allowed` / `gate.refused` events, plus a divergence report when `--mode live` differs from the recording.

Author-key revocation (SPEC-0198)

There is no top-level subcommand for this in the dispatcher today. The flow runs through the registry API:

```
curl -X POST $REG/api/skills/identities/<id>/revoke \
    -H "Authorization: Bearer $ADMIN_TOKEN" \
    -d '{"effective_at": "<ISO>", "rationale": "<why>"}'
```

A future `skillctl identity revoke <id>` is queued (SPEC-0198 §X). The bash form above is the canonical v1 flow.

Conventions in command output

- **Stderr** – all diagnostics, all error messages, structured `--verbose` lines.
- **Stdout** – machine-readable result. JSON when `--format json` or `--json`. Bare value (digest, fingerprint) for single-result calls.
- **Exit code** – branch on this. The numbered codes are stable; log strings are not.
- **No interactive prompts** by default. Destructive commands (`audit --cleanup`, `awareness reset`) require explicit confirmation flags.

Notable cross-command flags

Flag	Commands	Meaning
`--registry URL`	awareness, attest, install, verify, intent, propose, import-*	Override the registry endpoint
`--key PATH.priv`	sign, attest, awareness sync, intent, propose	Path to ed25519 private key (mode 0600)
`--tenant ID`	install, verify, run, invoke-replay	Pin operation to a tenant scope
`--allow-yellow`	install, verify	Lower the governance gate per-call (audited)
`--dry-run`	awareness sync/reset, import, import-public, propose	Preview without applying
`--verbose`	install, verify, run	Per-step structured stderr logs
`--json`	scan, audit, import-list, intent declare	Machine-readable output
`--home PATH`	install, verify	Override `$HOME` for tests

Worked examples (smallest viable runs)

E1 – Sign and verify a bundle locally (no registry)

```
# 1) Generate a keypair
skillctl keygen --out /tmp/tmp-key

# 2) Build a deterministic .skb
skillctl pack ~/.claude/skills/hello-skill --out /tmp/hello-0.1.0.skb

# 3) Sign it
skillctl sign /tmp/hello-0.1.0.skb --key /tmp/tmp-key.priv --identity-id id:test@local

# 4) Verify locally (no chain, just signature)
skillctl verify-sig /tmp/hello-0.1.0.skb --pubkey /tmp/tmp-key.pub
# -> "OK: signature verified"
```

E2 – Pin a registry, install, run

```
# 1) Pin
skillctl trust add \
    --registry https://aims.example.com/api/skills \
    --pubkey ~/.config/m3c/skill-trust-roots/aims-prod.pub \
    --id aims-prod

# 2) Install (full chain check)
skillctl install didactic-session@1.0.0 --tenant kup-berlin
```

```
# 3) Run (envelope-bounded)
skillctl run didactic-session --tenant kup-berlin
```

E3 – SKILLOR-style awareness sync

```
# 1) Inventory
skillctl scan --source claude --with-trust --format json > /tmp/inv.json

# 2) Dry-run
skillctl awareness sync \
    --inventory /tmp/inv.json \
    --registry https://aims.example.com/api/skills \
    --default-attest yellow \
    --session "skill-awareness/$(hostname -s)/$(date +%F)" \
    --dry-run

# 3) Confirm
skillctl awareness sync \
    --inventory /tmp/inv.json \
    --registry https://aims.example.com/api/skills \
    --default-attest yellow \
    --session "skill-awareness/$(hostname -s)/$(date +%F)" \
    --confirm

# 4) Verify round-trip
skillctl awareness verify --session "skill-awareness/$(hostname -s)/$(date +%F)"
```

E4 – Reviewer attestation

```
DIGEST="sha256:cd1d5c79..."
skillctl attest "$DIGEST" \
    --level green \
    --rationale "Activation gate passed; intent matches data_deps; smoke-tested locally." \
    --reviewer-id id:reviewer@m3c \
    --key ~/.claude/skillctl-keys/reviewer.key \
    --registry https://aims.example.com/api/skills
```

E5 – Import from a public hub (drafted SPEC-0201)

```
# 1) Without the pin -- refuses, prints required sha256
skillctl import-public skillhub://didactic-session
# -> "exit 4 pin_required: re-run with @sha256:abc123..."

# 2) With the pin
skillctl import-public skillhub://didactic-session@sha256:abc123... \
    --accept-license MIT

# 3) Inspect
skillctl import-list
cat ~/.cache/m3c/imports/abc123.../scan-report.json
```

```
# 4) Promote to candidate
skillctl propose didactic-session \
    --source ~/.cache/m3c/imports/abc123.../ \
    --derived-from skillhub://didactic-session@sha256:abc123... \
    --intent yellow \
    --rationale "imported from skillhub; pending reviewer"
```

What this manual does NOT cover

- **Server-side admin endpoints** (`/api/skills/identities`, `/api/tenants/*`, `/api/skills/audit-log`). See `aims-core/flask/modules/skill_registry/api.py` and the role playbook in USER-MANUAL.md §5.4 / §5.5.
- **The CISO Console UI** (`/v2/ciso?tenant=<id>`). See USER-MANUAL.md §5.3 and SPEC-0192.
- **The MCP skill-server tools** (`skill_install`, `skill_verify`, `skill_attest`, `skill_scan` for use directly from Claude Code). See SPEC-0122 and `m3c-tools/mcp-skill-server`.
- **The KafScale fan-out interface** (interface frozen in SPEC-0193; CLI surface lands once KafScale itself ships).
- **The federation protocol** for cross-pool republication (SPEC-0197 – Silver/Gold tier feature; not on the v1 single-pool path).

References

- USER-MANUAL.md – role-keyed playbooks, exit-code catalog, decision tree, troubleshooting
- SKILLOR-WORK/s1/USER-MANUAL.md – the original awareness session that the productized commands derive from
- CONCEPT.md – production architecture, semantic-graph proposition, productization
- ROLES.md – roles, interaction patterns, conflict zones
- IMPORT-AIRLOCK.md – operator brief for SPEC-0201
- RUNTIME-ENVELOPE.md – operator brief for SPEC-0202
- SPECs: -0188 · -0189 · -0194 · -0195 · -0196 · -0198 · -0201 · -0202
- Source of truth: `cmd/skillctl/main.go` on `s2/integration` (worktree `/Users/kamir/wt/spec-0189/s2-integration/`)